

Sviluppo Applicazioni Web

Simone Zenzaro

Cnr-Istituto di Linguistica Computazionale “Antonio Zampolli”

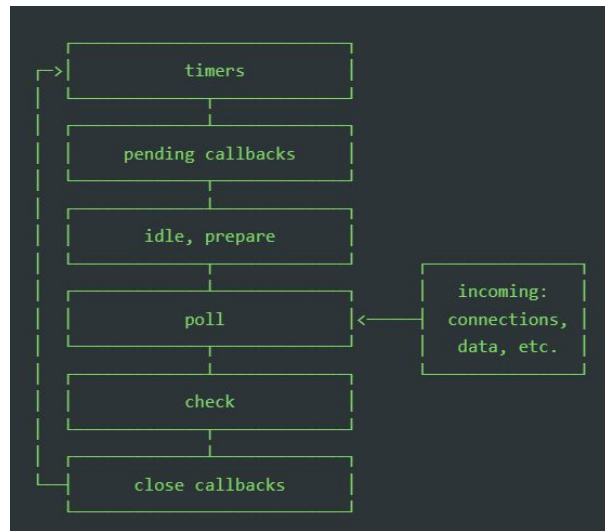
Email: simone.zenzaro@ilc.cnr.it

Ufficio: BM1, Ingresso 21

Servizi Web in Express

Node.js

- Node.js è un runtime Javascript open-source e cross-platform
- E' un runtime asincrono event-driven (vs thread based)
- Come per il runtime del browser anche Node.js inizializza ed esegue un Event Loop
- Ha avuto il merito di portare JavaScript sul server
- Enfasi su HTTP rende Node.js una buona scelta per la costruzione di servizi Web
- Fornisce una vasta API per la gestione di http, file system, funzionalità crittografiche, informazioni sul sistema operativo su cui viene eseguito e molto altro



Eseguire uno script con Node.js

- Creare un file javascript (es. index.js)
- Eseguire *node* *<file>* (es. node index.js)
- Per la gestione delle dipendenze si consiglia di sfruttare npm

Gestire le versioni di node

- Esistono diverse versioni di Node.js
- Per agevolare lo sviluppo in ambienti con molti progetti si può usare nvm
- nvm è un tool da riga di comando che permette di installare e usare diverse versioni di node
- nvm sta per Node Version Manager

```
$ nvm use 16
Now using node v16.9.1 (npm v7.21.1)
$ node -v
v16.9.1
$ nvm use 14
Now using node v14.18.0 (npm v6.14.15)
$ node -v
v14.18.0
$ nvm install 12
Now using node v12.22.6 (npm v6.14.5)
$ node -v
v12.22.6
```

Express

- Web framework minimalista per Node.js
- Un'applicazione Express consiste in una serie chiamate a ***middlewares***
- Express è minimale nel senso che l'applicazione ha funzionalità piuttosto limitate senza middleware
- L'intera libreria si basa su quattro oggetti:
 - Application
 - Request
 - Response
 - Router

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

Alternative ad Express

- Fastify
- Koa
- NestJS
- ...

Application

- E' un oggetto che rappresenta un'applicazione Express
- Si crea chiamando *express()*
- Ha metodi per:
 - fare il routing delle richieste HTTP
 - configurare i middleware
 - fare il rendering di HTML
 - registrare un template engine
- Il metodo `app.listen` si occupa del binding fra host e porta con le connessioni al servizio

Routing con Express

- `app.METHOD(PATH, HANDLER)`
- `app` è una istanza di `express`
- `METHOD` è un metodo HTTP
- `PATH` è un path sul server
- `HANDLER` è una funzione che viene eseguita quando viene fatta una richiesta al server che corrisponde al path
- `METHOD` può anche essere “all” nel caso si voglia gestire una richiesta indipendentemente dal metodo HTTP

```
app.get('/', (req, res) => {  
  res.send('Hello world!')  
})
```

Route path

- I route path possono essere
 - stringhe
 - string pattern
 - regex
- I path parameters possono essere definiti con il prefisso ":" ed il loro valore si troverà dentro *req.params*

```
/
```

```
/ricette
```

```
Route path: /ricette/:id/ingredienti/:idIngrediente
```

```
Request URL: /ricette/42/ingredienti/123
```

```
req.params: { "id": "42", "idIngrediente": "123" }
```

```
/ricette/:id.json
```

```
/ricette/:id.xml
```

```
/pages/:from-:to
```

Request

- E' un oggetto che rappresenta una richiesta HTTP
- Da accesso a tutte le informazioni relative alla richiesta
- **req.body**: contiene i dati relativi al body della richiesta (di default è undefined)
- **req.cookies**: usando il middleware cookie-parser è un oggetto che contiene i cookie
- **req.params**: contiene i path parameters della richiesta
- **req.query**: contiene i query parameters della richiesta
- **req.get**: restituisce il valore dell'header passato come parametro (case-insensitive)

Response

- E' un oggetto che rappresenta una risposta HTTP
- `res.cookie(name, value, [,options])`: imposta un cookie
- `res.clearCookie(name [, options])`: rimuove un cookie
- `res.download(path)`: trasferisce il file al path passato come parametro, il browser aprirà il prompt di download
- `res.end`: termina il processo di risposta
- `res.get`: restituisce il valore dell'header passato come parametro (case-insensitive)
- `res.json([body])`: invia una risposta JSON
- `res.redirect([status], path)`: ridireziona verso una URL
- `res.send([body])`: invia dei dati di risposta
- `res.status(code)`: imposta lo status code della risposta
- `res.set(field, [,value])`: imposta l'header della risposta
- `res.type(type)`: imposta il content type

Router

- E' una istanza isolata di middleware e routes
- Può essere pensato come una mini applicazione all'interno dell'applicazione
- Si comporta come un middleware (quindi si può usare con `app.use`)
- Utile per raggruppare middleware relativi a sotto-path su cui il router viene montato

Middleware

- Un middleware è una funzione che ha accesso a:
 - req: Request object
 - res: Response object
 - next: la funzione del prossimo middleware
- Un middleware può:
 - Eseguire del codice
 - Modificare req e res
 - Terminare il ciclo di richiesta e risposta
 - Chiamare il prossimo middleware
- Esistono diversi tipi di middleware:
 - A livello di applicazione
 - A livello di router
 - Gestione degli errori
 - Built-in
 - Di terze parti

Middleware a livello di applicazione

- Utilizzare `app.use` o `app.METHOD`
- Il codice relativo al middleware viene eseguito:
 - ad ogni richiesta (`app.use`)
 - ogni volta che il path corrisponde al parametro (`app.use(path)`)
 - ogni volta che path e metodo corrispondono (`app.METHOD`)
- `next` è la chiamata al prossimo middleware
- se non viene chiamata `next`, il ciclo di chiamate a middleware termina

```
const express = require('express');
const app = express();
app.use((req, res, next) => {
  console.log('App middleware call')
  next()
});

const express = require('express');
const app = express();
app.use('/my-path', (req, res, next) => {
  console.log('App middleware relativo al path')
  next()
});
app.get('/my-path', (req, res, next) => {
  console.log('App middleware relativo metodo e al path')
  next()
});
```

Middleware a livello di Router

- I middleware a livello di Router sono del tutto simili a quelli al livello dell'applicazione
- Invece di legarli ad una istanza di Application si legano ad una istanza di Router

```
const express = require('express')
const app = express()
const router = express.Router()

router.use((req, res, next) => {
  console.log('Router middleware')
  next()
});
```


Middleware per la gestione degli errori

- Definiscono delle funzioni che gestiscono errori
- Sono come gli altri middleware ma hanno un parametro aggiuntivo *err*
- Express fornisce già un error handler di default che aggiunge lo status 500 Internal Server Error

```
app.use((err, req, res, next) => {  
  console.error(err.stack)  
  res.status(500).send('Something went wrong')  
})
```

Built-in middleware

- Express fornisce tre middleware di default
- `express.static`: utile per servire file statici
- `express.json`: parse le richieste considerando il loro payload come JSON
- `express.urlencoded`: parse le richieste con payload URL-encoded

```
const express = require('express');  
const app = express();  
  
app.use(express.json());
```

Middleware di terze parti

- Qualsiasi funzione o pacchetto che implementa un middleware può essere usato come middleware

```
// $ npm install cookie-parser  
const express = require('express')  
const app = express()  
const cookieParser = require('cookie-parser')  
  
app.use(cookieParser())
```

CORS Middleware

```
let express = require('express')
let cors = require('cors')

const app = express();
const port = 2345;

app.use(cors())
...

//-----

app.use(cors({
  origin: ['*', 'http://allowed-host.edu'],
  methods: ['PUT', 'POST'],
})))
```

Chiamare un servizio dal client: Fetch API

- La Fetch API è un'interfaccia per il fetching di risorse
- Fornisce una definizione per le richieste (Request) e le risposte (Response)
- Definisce anche i concetti di CORS
- il metodo ***fetch(path)*** si trova nello scope globale
 - il parametro path definisce il path alla risorsa
 - restituisce una Promise che risulta in un oggetto Response
- Se il contenuto della risposta è un json, si può recuperare con il metodo `response.json()`

```
fetch("http://api.dev/v1/todos/42")  
  .then((response) => response.json())  
  .then((data) => console.log(data));
```

Opzioni per fetch

- **method**: il metodo HTTP per la richiesta
- **headers**: l'insieme di header per la richiesta
- **body**: i dati che accompagnano la richiesta
- **redirect**: definisce il comportamento in caso di risposta redirect, una fra *follow*, *error*, *manual*
- **credentials**: definisce il comportamento rispetto alle credenziali, una fra *omit*, *same-origin*, *include*
- **cache**: indica il modo in cui la richiesta interagisce con la cache del browser, uno fra *default*, *no-store*, *reload*, *no-cache*, *force-cache*, *only-if-cached*

```
fetch(url, {  
  method: "POST",  
  mode: "cors",  
  cache: "no-cache",  
  credentials: "same-origin",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  redirect: "follow",  
  body: JSON.stringify(data),  
}).then((response) => response.json());
```

Esercizio

- Creare una REST API per la gestione dei TODO in express
- Chiamare la REST API dalla TODO Web App